

Lecture 3 - OOP

Dynamic Array Implementation.

TElem = void *
(generic pointers)

DynamicArray.h

#pragma once
(#include "Planet.h")

typedef void* TElem; → could be

typedef void (*destroyFct)(TElem)

pointer to function

char *

Planet *

int *

type of a function that receives a generic element and it destroys it.

typedef struct {

int capacity, size;

TElem* elems; → void** elems

destroyFct destructionOperation;

} DynamicArray;

DynamicArray* createArray (int cap, destroyFct op);

void destroyDynamicArray (DynamicArray* arr);

void add (DynamicArray* arr, TElem elem);

DynamicArray.c

#include "DynamicArray.h"

#include <stdlib.h>

DynamicArray* createArray (int cap, destroyFct op)

{

DynamicArray* arr = malloc (sizeof (DynamicArray));

if (arr == NULL)

return NULL;

arr → capacity = cap;

arr → size = 0;

arr → destructionOperation = op;

arr → elems = malloc (sizeof (TElem) * cap);

if (arr → elems == NULL)

{ free (arr);

return NULL;

}

return arr;

}

void destroyDynamicArray (DynamicArray* arr)

{ if (arr == NULL)

return;

arr → capacity = 0;

arr → elems = NULL;

arr → size = 0;

arr → destructionOperation = NULL;

arr → elems = NULL;

arr → size = 0;

arr → capacity = 0;

arr → elems = NULL;

arr → size = 0;

arr → capacity = 0;

arr → elems = NULL;

arr → size = 0;

arr → capacity = 0;

arr → elems = NULL;

arr → size = 0;

arr → capacity = 0;

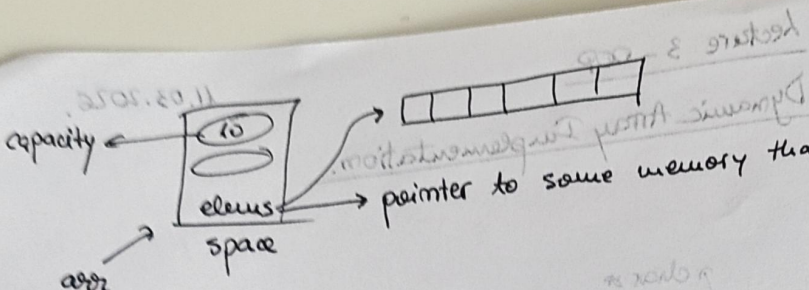
arr → elems = NULL;

arr → size = 0;

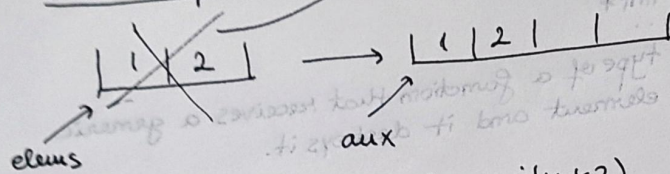
arr → capacity = 0;

arr → elems = NULL;

we want to destroy these objects



REALLOC operation:



- I. double the capacity ($capacity * 2$)
- II. allocate new space (aux)
- III. copy existing elements
- IV. free old space
- V. make new pointer (elems) point to the new memory (aux)

→ actual pointer is on the STACK.
 → the memory the pointer is pointing to is on HEAP.

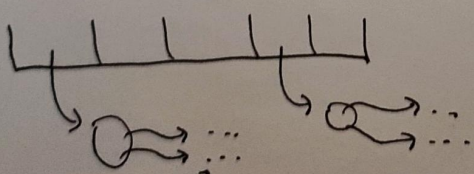
```

[c]: void resize(DynamicArray* arr) {
    if (arr == NULL)
        return;
    arr->capacity *= 2;
    T* aux = malloc(sizeof(T) * arr->capacity);
    for (int i = 0; i < arr->size; i++)
        aux[i] = arr->elems[i];
    free(arr->elems);
    arr->elems = aux;
}
  
```

```

T* aux = realloc(arr->elems, arr->capacity * sizeof(T));
if (aux == NULL)
    return;
arr->elems = aux;
  
```

! Array of pointers to void → point to anything (makes more generic).
 Dynamic array that stores pointers, not objects.



we want to destroy those as well